

[Lab 5] [cheatsheet]

Task 1 | Global Thresholding (3 values) + Adaptive

```
import cv2
import matplotlib.pyplot as plt

# Load image in grayscale
img = cv2.imread("sudoku.png", cv2.IMREAD_GRAYSCALE)

# Global thresholding
_, g1 = cv2.threshold(img, 80, 255, cv2.THRESH_BINARY)
_, g2 = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)
_, g3 = cv2.threshold(img, 180, 255, cv2.THRESH_BINARY)

# Adaptive thresholding
a = cv2.adaptiveThreshold(
    img,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

# Display results
fig, ax = plt.subplots(2, 3, figsize=(12, 8))

ax[0, 0].imshow(img, cmap="gray")
ax[0, 0].set_title("Original")

ax[0, 1].imshow(g1, cmap="gray")
ax[0, 1].set_title("Global T=80")

ax[0, 2].imshow(g2, cmap="gray")
ax[0, 2].set_title("Global T=128")

ax[1, 0].imshow(g3, cmap="gray")
ax[1, 0].set_title("Global T=180")

ax[1, 1].imshow(a, cmap="gray")
ax[1, 1].set_title("Adaptive")

for x in ax.flat:
    x.axis("off")

ax[1, 2].axis("off")

plt.tight_layout()
plt.savefig("task1_result.png")
plt.show()
```

Task 2 | Adaptive Thresholding (Mean / Gaussian)

```
import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("sudoku.png", cv2.IMREAD_GRAYSCALE)

# Mean adaptive thresholding
m1 = cv2.adaptiveThreshold(
    img, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    7, 2
)

m2 = cv2.adaptiveThreshold(
    img, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    11, 2
)

m3 = cv2.adaptiveThreshold(
    img, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    21, 2
)

# Gaussian adaptive thresholding
g1 = cv2.adaptiveThreshold(
    img, 255,
```

```
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11, 2
)

g2 = cv2.adaptiveThreshold(
    img, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11, 5
)

g3 = cv2.adaptiveThreshold(
    img, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11, 10
)

# Display
fig, ax = plt.subplots(2, 4, figsize=(14, 7))

ax[0, 0].imshow(img, cmap="gray")
ax[0, 0].set_title("Original")

ax[0, 1].imshow(m1, cmap="gray")
ax[0, 1].set_title("Mean B=7 C=2")

ax[0, 2].imshow(m2, cmap="gray")
ax[0, 2].set_title("Mean B=11 C=2")

ax[0, 3].imshow(m3, cmap="gray")
ax[0, 3].set_title("Mean B=21 C=2")

ax[1, 0].imshow(g1, cmap="gray")
ax[1, 0].set_title("Gaussian B=11 C=2")

ax[1, 1].imshow(g2, cmap="gray")
ax[1, 1].set_title("Gaussian B=11 C=5")

ax[1, 2].imshow(g3, cmap="gray")
ax[1, 2].set_title("Gaussian B=11 C=10")

ax[1, 3].axis("off")

for x in ax.flat:
    x.axis("off")

plt.tight_layout()
plt.savefig("task2_result.png")
plt.show()
```

Task 3 | Otsu's Thresholding

```
import cv2
import matplotlib.pyplot as plt

# Load image in grayscale
img = cv2.imread("coins.png", cv2.IMREAD_GRAYSCALE)

# Otsu thresholding
t1, o1 = cv2.threshold(
    img,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# Change image contrast
img2 = cv2.convertScaleAbs(img, alpha=1.5, beta=0)

# Otsu thresholding again
t2, o2 = cv2.threshold(
    img2,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# Display results
fig, ax = plt.subplots(2, 3, figsize=(12, 8))

ax[0, 0].imshow(img, cmap="gray")
```

```

ax[0, 0].set_title("Original")
ax[0, 0].axis("off")

ax[0, 1].hist(img.ravel(), 256, [0, 256])
ax[0, 1].set_title("Original Histogram")

ax[0, 2].imshow(o1, cmap="gray")
ax[0, 2].set_title("Otsu")
ax[0, 2].axis("off")

ax[1, 0].imshow(img2, cmap="gray")
ax[1, 0].set_title("Changed Contrast")
ax[1, 0].axis("off")

ax[1, 1].hist(img2.ravel(), 256, [0, 256])
ax[1, 1].set_title("New Histogram")

ax[1, 2].imshow(o2, cmap="gray")
ax[1, 2].set_title("Otsu After Contrast")
ax[1, 2].axis("off")

plt.tight_layout()
plt.savefig("task3_result.png")
plt.show()

# Record thresholds
print("Original Otsu threshold:", t1)
print("Changed contrast Otsu threshold:", t2)

```

Task 4 | Color-Based Segmentation (HSV)

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("smarties.png")

# Convert BGR to HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Restrictive mask
low1 = np.array([20, 100, 100])
up1 = np.array([35, 255, 255])

mask1 = cv2.inRange(hsv, low1, up1)

# Final mask
low2 = np.array([15, 50, 50])
up2 = np.array([40, 255, 255])

mask2 = cv2.inRange(hsv, low2, up2)

# Extract selected color
result = cv2.bitwise_and(img, img, mask=mask2)

# Convert BGR to RGB for matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
result_rgb = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)

# Display
fig, ax = plt.subplots(2, 2, figsize=(10, 8))

ax[0, 0].imshow(img_rgb)
ax[0, 0].set_title("Original")

ax[0, 1].imshow(mask1, cmap="gray")
ax[0, 1].set_title("Restrictive Mask")

ax[1, 0].imshow(mask2, cmap="gray")
ax[1, 0].set_title("Final Mask")

ax[1, 1].imshow(result_rgb)
ax[1, 1].set_title("Extracted Yellow")

for x in ax.flat:
    x.axis("off")

plt.tight_layout()
plt.savefig("task4_result.png")
plt.show()

```

Task 5 | Canny Edge Detection

```

import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("smarties.png")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Canny edge detection
e1 = cv2.Canny(gray, 50, 100)
e2 = cv2.Canny(gray, 100, 200)
e3 = cv2.Canny(gray, 150, 250)

# Display
fig, ax = plt.subplots(2, 2, figsize=(10, 8))

ax[0, 0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
ax[0, 0].set_title("Original")

ax[0, 1].imshow(e1, cmap="gray")
ax[0, 1].set_title("Canny 50,100")

ax[1, 0].imshow(e2, cmap="gray")
ax[1, 0].set_title("Canny 100,200")

ax[1, 1].imshow(e3, cmap="gray")
ax[1, 1].set_title("Canny 150,250")

for x in ax.flat:
    x.axis("off")

plt.tight_layout()
plt.savefig("task5_result.png")
plt.show()

```

Task 6 | Region Growing

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("brain_mri.png", cv2.IMREAD_GRAYSCALE)

# Region growing function
def region_growing(img, seed, threshold):
    mask = np.zeros_like(img, dtype=np.uint8)
    stack = [seed]

    seed_value = int(img[seed])

    while stack:
        x, y = stack.pop()

        if x < 0 or x >= img.shape[0] or y < 0 or y >= img.shape[1]:
            continue

        if mask[x, y] == 0:
            if abs(int(img[x, y]) - seed_value) < threshold:
                mask[x, y] = 255

                stack.append((x + 1, y))
                stack.append((x - 1, y))
                stack.append((x, y + 1))
                stack.append((x, y - 1))

    return mask

# Two seed points
s1 = (100, 100)
s2 = (150, 150)

# Three thresholds
t1 = 20
t2 = 40
t3 = 60

# Generate regions
r1 = region_growing(img, s1, t1)
r2 = region_growing(img, s1, t2)
r3 = region_growing(img, s1, t3)

```

```

r4 = region_growing(img, s2, t1)
r5 = region_growing(img, s2, t2)
r6 = region_growing(img, s2, t3)

# Display
fig, ax = plt.subplots(2, 4, figsize=(14, 8))

ax[0, 0].imshow(img, cmap="gray")
ax[0, 0].set_title("Original")

ax[0, 1].imshow(r1, cmap="gray")
ax[0, 1].set_title("Seed 1, T=20")

ax[0, 2].imshow(r2, cmap="gray")
ax[0, 2].set_title("Seed 1, T=40")

ax[0, 3].imshow(r3, cmap="gray")
ax[0, 3].set_title("Seed 1, T=60")

ax[1, 0].imshow(r4, cmap="gray")
ax[1, 0].set_title("Seed 2, T=20")

ax[1, 1].imshow(r5, cmap="gray")
ax[1, 1].set_title("Seed 2, T=40")

ax[1, 2].imshow(r6, cmap="gray")
ax[1, 2].set_title("Seed 2, T=60")

ax[1, 3].axis("off")

for x in ax.flat:
    x.axis("off")

plt.tight_layout()
plt.savefig("task6_result.png")
plt.show()

```

Task 7 | Watershed

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("coins.png")

# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Thresholding
_, thresh = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Noise removal
kernel = np.ones((5, 5), np.uint8)
opening = cv2.morphologyEx(
    thresh,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Sure background
sure_bg = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Distance transform
dist = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Normalize distance transform for display
dist_show = cv2.normalize(
    dist,
    None,
    0,

```

```

    255,
    cv2.NORM_MINMAX
)

# Sure foreground
_, sure_fg = cv2.threshold(
    dist,
    0.2 * dist.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

# Marker labeling
_, markers = cv2.connectedComponents(sure_fg)

# Background marker
markers = markers + 1

# Unknown region = 0
markers[unknown == 255] = 0

# Watershed
img2 = img.copy()
cv2.watershed(img2, markers)

# Mark boundaries
img2[markers == -1] = [0, 0, 255]

# Convert images for matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
result_rgb = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)

# Display
fig, ax = plt.subplots(2, 4, figsize=(14, 8))

ax[0, 0].imshow(img_rgb)
ax[0, 0].set_title("Original")
ax[0, 0].axis("off")

ax[0, 1].imshow(thresh, cmap="gray")
ax[0, 1].set_title("Threshold")
ax[0, 1].axis("off")

ax[0, 2].imshow(sure_bg, cmap="gray")
ax[0, 2].set_title("Sure Background")
ax[0, 2].axis("off")

ax[0, 3].imshow(dist_show, cmap="gray")
ax[0, 3].set_title("Distance Transform")
ax[0, 3].axis("off")

ax[1, 0].imshow(sure_fg, cmap="gray")
ax[1, 0].set_title("Sure Foreground")
ax[1, 0].axis("off")

ax[1, 1].imshow(unknown, cmap="gray")
ax[1, 1].set_title("Unknown Region")
ax[1, 1].axis("off")

ax[1, 2].imshow(result_rgb)
ax[1, 2].set_title("Final Watershed")
ax[1, 2].axis("off")

ax[1, 3].axis("off")

plt.tight_layout()
plt.savefig("task7_result.png")
plt.show()

```

Task 8 | Watershed Tuning

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("coins.png")

# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Thresholding
_, thresh = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Noise removal
kernel = np.ones((5, 5), np.uint8)
opening = cv2.morphologyEx(
    thresh,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Sure background
sure_bg = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Distance transform
dist = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Four distance thresholds
ts = [0.1, 0.2, 0.3, 0.4]

results = []

for t in ts:

    # Sure foreground
    _, sure_fg = cv2.threshold(
        dist,
        t * dist.max(),
        255,
        0
    )

    sure_fg = np.uint8(sure_fg)

    # Number of foreground markers
    n, markers = cv2.connectedComponents(sure_fg)
    fg = n - 1

    # Unknown region
    unknown = cv2.subtract(sure_bg, sure_fg)

    # Marker labeling
    markers = markers + 1
    markers[unknown == 255] = 0

    # Watershed
    result = img.copy()
    cv2.watershed(result, markers)

    # Number of separated regions
    labels = np.unique(markers[markers > 1])
    regions = len(labels)

    # Draw watershed boundaries
    result[markers == -1] = [0, 0, 255]

    results.append((t, fg, regions, result))

# Print experiment table
print("Threshold | Foreground | Regions")
```

```
for t, fg, regions, result in results:
    print(t, " | ", fg, " | ", regions)

# Display
fig, ax = plt.subplots(2, 3, figsize=(12, 8))

ax[0, 0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
ax[0, 0].set_title("Original")
ax[0, 0].axis("off")

for i, (t, fg, regions, result) in enumerate(results):
    r = i // 2
    c = (i % 2) + 1

    ax[r, c].imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
    ax[r, c].set_title(
        "T=" + str(t) +
        " | FG=" + str(fg) +
        " | R=" + str(regions)
    )
    ax[r, c].axis("off")

plt.tight_layout()
plt.savefig("task8_result.png")
plt.show()
```

Task 9 | K-Means Segmentation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("dog.jpeg")

# Convert image to pixel list
pixels = img.reshape((-1, 3))
pixels = np.float32(pixels)

# K-Means function
def segment(img, k):
    pixels = img.reshape((-1, 3))
    pixels = np.float32(pixels)

    criteria = (
        cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
        100,
        0.2
    )

    _, labels, centers = cv2.kmeans(
        pixels,
        k,
        None,
        criteria,
        10,
        cv2.KMEANS_RANDOM_CENTERS
    )

    centers = np.uint8(centers)
    result = centers[labels.flatten()]

    return result.reshape(img.shape)

# Segment for K = 2, 4, 6
k2 = segment(img, 2)
k4 = segment(img, 4)
k6 = segment(img, 6)

# Convert BGR to RGB
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
k2 = cv2.cvtColor(k2, cv2.COLOR_BGR2RGB)
k4 = cv2.cvtColor(k4, cv2.COLOR_BGR2RGB)
k6 = cv2.cvtColor(k6, cv2.COLOR_BGR2RGB)

# Display
fig, ax = plt.subplots(1, 4, figsize=(16, 5))

ax[0].imshow(img)
ax[0].set_title("Original")

ax[1].imshow(k2)
ax[1].set_title("K = 2")
```

```

ax[2].imshow(k4)
ax[2].set_title("K = 4")

ax[3].imshow(k6)
ax[3].set_title("K = 6")

for x in ax.flat:
    x.axis("off")

plt.tight_layout()
plt.savefig("task9_result.png")
plt.show()

```

Task 10 | Final: HSV + Canny + K-Means

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("smarties.png")

# -----
# 1. HSV SEGMENTATION
# -----

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

low = np.array([15, 50, 50])
up = np.array([40, 255, 255])

mask = cv2.inRange(hsv, low, up)

hsv_result = cv2.bitwise_and(img, img, mask=mask)

# -----
# 2. CANNY EDGE DETECTION
# -----

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 100, 200)

# -----
# 3. K-MEANS
# -----

pixels = img.reshape((-1, 3))
pixels = np.float32(pixels)

```

```

criteria = (
    cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
    100,
    0.2
)

_, labels, centers = cv2.kmeans(
    pixels,
    4,
    None,
    criteria,
    10,
    cv2.KMEANS_RANDOM_CENTERS
)

centers = np.uint8(centers)

kmeans_result = centers[labels.flatten()]
kmeans_result = kmeans_result.reshape(img.shape)

# -----
# DISPLAY
# -----

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
hsv_rgb = cv2.cvtColor(hsv_result, cv2.COLOR_BGR2RGB)
kmeans_rgb = cv2.cvtColor(kmeans_result, cv2.COLOR_BGR2RGB)

fig, ax = plt.subplots(1, 4, figsize=(16, 5))

ax[0].imshow(img_rgb)
ax[0].set_title("Original")

ax[1].imshow(hsv_rgb)
ax[1].set_title("HSV Segmentation")

ax[2].imshow(edges, cmap="gray")
ax[2].set_title("Canny")

ax[3].imshow(kmeans_rgb)
ax[3].set_title("K-Means K=4")

for x in ax.flat:
    x.axis("off")

plt.tight_layout()
plt.savefig("task10_result.png")
plt.show()

```